Theoretical Computer Science Exam, February 13, 2019

The exam consists of **4 exercises**. Available time: 1 hr 40 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME …….. L.NAME …….. MATRICOLA ……

I have passed the midterm exam and I intend to skip the first part (Ex.1-2) (select one): **YES NO**

Exercise 1 (8 pts)

1. Design a least general grammar, and a least powerful automaton, that respectively generate and accept the following language

$$L_1 = \{\, a^m\, b^k\, c^n\, d \;\mid\; m > k + n \,\}$$

2. Design a least general grammar, and a least powerful automaton, that respectively generate and accept the following language

$$L_2 = \{\, a^m\, b^k\, c^n \;\mid\; m > 0 \;\wedge\; n > 0 \;\wedge\; m + n \le k \le 2m + 2n \,\}$$

Exercise 2 (8 pts)

Specify, by means of suitable pre- and post-conditions, written in the usual first-order logic for program specification, a subprogram ***searcheck*(*v*, *n*, *e*, *c*)** where: ***v*** and ***n*** are input parameters representing, respectively, an array of integers and the number of its elements; ***e*** is an integer input parameter, and ***c*** is an output integer parameter**,** whose value is determined according to the following clauses (assume that indices in the array start from 0).

- ***c*** = -1 iff ***e*** is not present in ***v*** ;
- ***c*** = 0 iff ***e*** is present in ***v*** with exactly one occurrence;
- if there are at least 2 distinct occurrences of ***e*** in ***v***, then
  - ***c*** = 1 if for every two distinct occurrences of ***e*** inside ***v*** all the elements of ***v*** included between them are > ***e*** (example: ***e*** = 4 and ***v***=[1, 2, 3, 4, 7, 8, 9, 5, 4, 2, 1]);
  - ***c*** = 2 if for every two distinct occurrences of ***e*** inside ***v*** all the elements of ***v*** included between them are < ***e*** (example: ***e*** = 4 and ***v***=[1, 2, 3, 4, 1, 2, -1, -2, 4, 2, 1]);
  - ***c*** = 3 if for every two distinct occurrences of ***e*** inside ***v*** some (i.e., at least one) element of ***v*** included between them is < ***e*** and some (i.e., at least one) element of ***v*** included between them is > ***e*** (example: ***e*** = 4 and ***v***=[1, 2, 3, 4, 1, 6, -1, -2, 4, 2, 1]).

Say if the above specification is *complete*, i.e., if the value of the output parameter is defined for all possible values of the input parameters.

Exercise 3 (8 pts)

Consider the following set S_1

$$S_1 = \{\, y \mid \forall n \text{ the Turing machine } M_y \text{ does not accept any string of length } n \,\}$$

State, providing suitable clear and precise justifications, whether set S_1 is decidable or semidecidable.

Consider next the following set S_2

$$S_2 = \{\, y \mid \forall n \text{ the Turing machine } M_y \text{ accepts some string of length } n \text{ in a number of steps} \le n \,\}$$

State, providing suitable clear and precise justifications, whether at least one of the two sets, S_2 and $\neg S_2$, is semidecidable (remember that $\neg S_2$ denotes the complement of set S_2).

Exercise 4 (8 pts)

Design a Turing machine that accepts ***in real time*** the language

$$\{\, x\#y\# \mid x, y \in \{a, b, c, d\}^{+} \text{ and } x \text{ is an anagram of } y \,\}$$

(Let us remind that a string x is an anagram of a string y if the two strings contain the same letters, each letter having in each string the same number of occurrences, e.g., as in *badacdbbd* and *bdcadbadb*).

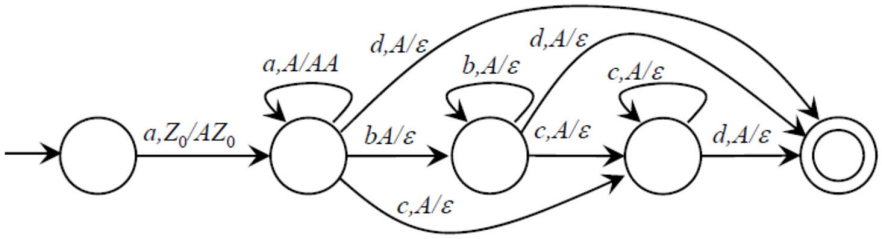
Sketch a RAM machine that accepts the same language, and compare the time and space complexity of the two machines, according to both the constant and the logarithmic cost criterion.

Solutions

Exercise 1

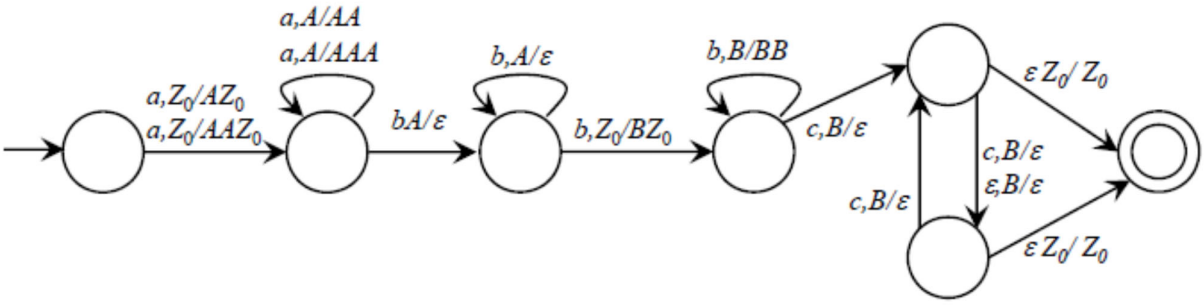
1. The language is context free. Notice that both k and n can be null, while m is strictly positive.
   $S \rightarrow A\ d$
   $A \rightarrow a\ A\ c \mid a\ B$
   $B \rightarrow a\ B\ b \mid a\ B \mid \varepsilon$

   A deterministc PDA suffices.



2. The language is context free.
   $S \rightarrow A\ C$
   $A \rightarrow a\ A\ b \mid a\ A\ bb \mid ab \mid abb$
   $C \rightarrow b\ C\ c \mid b\ b\ C\ c \mid bc \mid bbc$

   A nondeterministic PDA is necessary because of the many values of k that are compatible with any given pair of values m and n.



Exercise 2

Pre: $n > 0$

Post:

$$(c = -1 \leftrightarrow \neg\exists i \, (\, 0 \le i < n \wedge v[i] = e \,) \,) \wedge \qquad (-1)$$

$$(c = 0 \leftrightarrow \exists i \, (\, 0 \le i < n \wedge v[i] = e \,) \wedge \neg\exists i \, \exists j \, (0 \le i < j < n \wedge v[i] = e \wedge v[j] = e \,) \,) \wedge \qquad (0)$$

$$(\, (\exists i \, \exists j \, (0 \le i < j < n \;\wedge\; v[i] = e \wedge v[j] = e \,) \rightarrow$$

$$(\; \forall i \, \forall j \, (0 \le i < j < n \;\wedge\; v[i] = e \wedge v[j] = e \;\rightarrow\; \forall k \, (i < k < j \rightarrow v[k] > e \,) \,) \;\rightarrow\; c = 1 \,) \wedge \qquad (1)$$

$$(\; \forall i \, \forall j \, (0 \le i < j < n \;\wedge\; v[i] = e \wedge v[j] = e \;\rightarrow\; \forall k \, (i < k < j \rightarrow v[k] < e \,) \,) \;\rightarrow\; c = 2 \,) \wedge \qquad (2)$$

$$(\; \forall i \, \forall j \, (\qquad (3)$$

$$0 \le i < j < n \;\wedge\; v[i] = e \wedge v[j] = e$$

$$\rightarrow$$

$$\exists k \, (i < k < j \wedge v[k] < e \,) \wedge \exists k \, (i < k < j \wedge v[k] > e \,)$$

$$)$$

$$\rightarrow \; c = 3 \;)$$

$$)$$

The value of the output parameter ***c*** is not uniquely defined in the case, for instance, where there exist two consecutive occurrences of ***e*** inside ***v***. In that case the sub-clauses defining the value of ***c*** in lines (1) and (2) are both (vacuously) satisfied. If there are three or more consecutive occurrences of ***e*** inside ***v***, then none of the sub-clauses defining the value of ***c*** in lines (-1), (0), (1), (2), and (3) is satisfied. Example for both the above described cases: ***e*** = 4 and ***v*** = [1, 2, 3, 4, 4, 2, -1, 4, 4, 4, 1].

Exercise 3

Set S_1 is undecidable, which can be justified based on the Rice Theorem.

Let us consider the complement set $\neg S_1$

$$\neg S_1 = \{ \, y \;\mid\; \exists n \text{ such that the Turing machine } M_y \text{ accepts some string of length } n \; \}$$

S_1 is the set of TM with empty accepted language, hence $\neg S_1$ is the set of TM with nonempty accepted language, and such set is obviously semidecidable by dovetailing simulation. Therefore S_1 is not semidecidable.

Let us consider the complement set $\neg S_2$

$$\neg S_2 = \{ \, y \mid \exists n \text{ such that Turing machine } M_y \text{ does not accept any string of length } n \text{ in } \le n \text{ steps} \}$$

A more formal definition of set $\neg S_2$ is reported here below (where $T_{My}(x)$ denotes the time complexity of machine M_y on input x)

$$\neg S_2 = \{ \, y \mid \exists n \, \neg\exists x \, (\, |x|=n \wedge f_y(x) \neq \perp \wedge T_{My}(x) \le n \,) \,) \; \}$$

$\neg S_2$ is semidecidable (by means of dovetail simulation): for a given machine y an exhaustive simulation, over all possible integers n, of the computation of machine M_y for all possible inputs x of length $|x|=n$ can be carried out to check if the computation terminates in $\le n$ steps; then if a value n for which a string with such a computation is found then y can be stated to belong to $\neg S_2$.

Notice that a similar reasoning *cannot* be applied to S_2 because of the universal quantification over variable n in the definition of S_2 (a simulation cannot check that the required property is satisfied *for all* the infinite possible values of n).

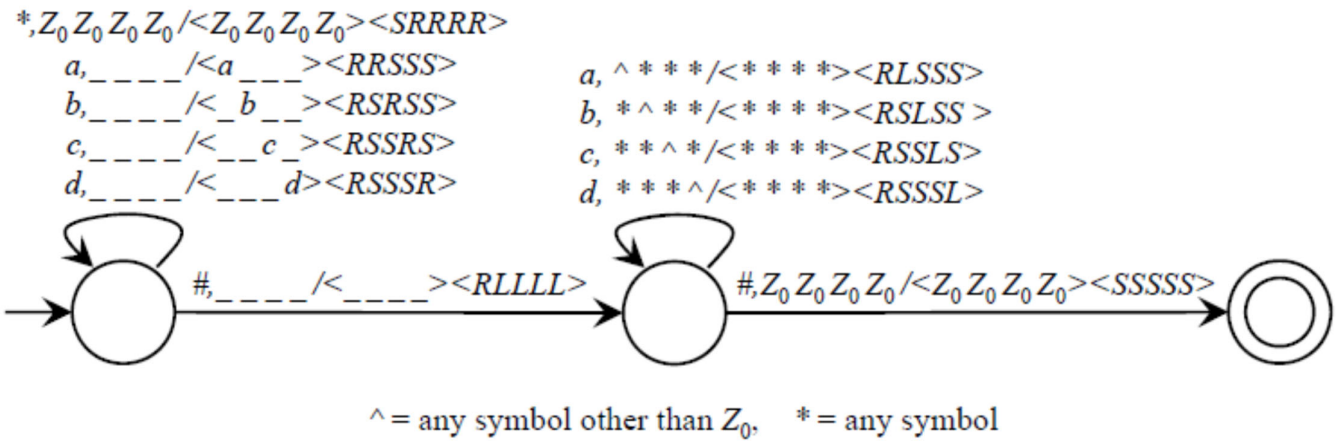
Notice also that the Rice therem cannot be applied to S_2 or to $\neg S_2$ because these sets are characterized by properties concerning the computation of a TM, not the computed fuction.

Exercise 4

For the Turing machine below the space complexity for an input string w, with $|w|=n$, is

- $S_T(n) = n$ if $w \notin L$
- $S_T(n) = (n\text{-}2)/2$ if $w \in L$

hence it is $\Theta(n)$.



The RAM machine scans the input string while counting the occurrences of the various symbols in the substrings x and y, and finally compares them. Its time complexty according to the constant criterion is $\Theta(n)$, but adopting the logarithmic cost criterion the time complexity is greater than n, because of the steps necessary for computing and comparing the values of the counters: it is $\Theta(n\ log(n))$, therefore worse than that of the Turing machine. The space complexity is determined by the variables storing the counters, therefore it is constant with the constant criterion, and $\Theta(log(n))$ with the logarithmic cost criterion.

The advantage of the Turing machine over the RAM machine concerning the time complexity is obtained at the price of an increased space complexity due to the unary encoding of the counters in the TM tapes: a typical space-time tradeoff.